

https://docs.ansible.com/projects/ansible/latest/getting_started/introduction.html

Ansible provides open-source automation that reduces complexity and runs everywhere. Using Ansible lets you automate virtually any task. Here are some common use cases for Ansible:

- Eliminate repetition and simplify workflows
- Manage and maintain system configuration
- Continuously deploy complex software
- Perform zero-downtime rolling updates

Ansible uses simple, human-readable scripts called playbooks to automate your tasks. You declare the desired state of a local or remote system in your playbook. Ansible ensures that the system remains in that state.

Agent-less architecture

Low maintenance overhead by avoiding the installation of additional software across IT infrastructure.

Simplicity

Automation playbooks use straightforward YAML syntax for code that reads like documentation. Ansible is also decentralized, using SSH with existing OS credentials to access remote machines.

Scalability and flexibility

Easily and quickly scale the systems you automate through a modular design that supports a large range of operating systems, cloud platforms, and network devices.

Idempotence and predictability

When the system is in the state your playbook describes, Ansible does not change anything, even if the playbook runs multiple times.

So, Ansible is: An automation tool that runs commands on remote machines over SSH

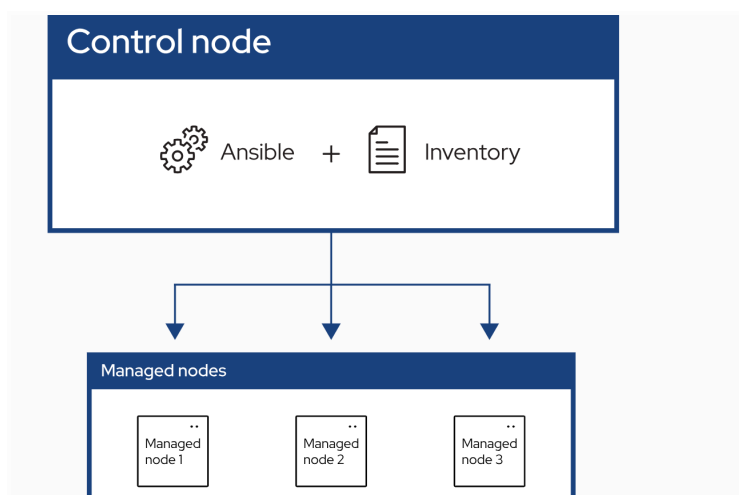
Feature	Ansible	Puppet	Chef	SaltStack	
Setup	Easy	Complex	Complex	Medium	
Agent needed?	✗ No	✓ Yes	✓ Yes	Both	
Language	YAML	DSL	Ruby	YAML/Python	
Learning curve	Low	Medium	High	Medium	
Speed	Medium	Medium	Medium	Fast	
Best for	Beginners / DevOps	Large infra	Advanced users	Real-time systems	

YAML is a [human-readable data serialization language](#). It is commonly used for [configuration files](#) and in applications where data is being stored or transmitted. YAML targets many of the same communications applications as [Extensible Markup Language](#) (XML) but has a minimal [syntax](#) that intentionally differs from [Standard Generalized Markup Language](#) (SGML).

Control node A system on which Ansible is installed. You run Ansible commands such as `ansible` or `ansible-inventory` on a control node.

Inventory A list of managed nodes that are logically organized. You create an inventory on the control node to describe host deployments to Ansible.

Managed node A remote system, or host, that Ansible controls.



Docker networking :

When creating a network, IPv4 address allocation is enabled by default, it can be disabled using `--ipv4=false`. IPv6 address allocation can be enabled using `--ipv6`.

```
$ docker network create --ipv6 --ipv4=false v6net
```

By default, the container gets an IP address for every Docker network it attaches to. A container receives an IP address out of the IP subnet of the network. The Docker daemon performs dynamic subnetting and IP address allocation for containers. Each network also has a default subnet mask and gateway.

You can connect a running container to multiple networks, either by passing the `--network` flag multiple times when creating the container, or using the `docker network connect` command for already running containers. In both cases, you can use the `--ip` or `--ip6` flags to specify the container's IP address on that particular network.

<https://docs.docker.com/engine/network/>

`docker network create ansible-net` : creates a virtual LAN inside docker